MeterMind: Interview Answers

These are answers to 45 likely interview questions, written the way I'd say them out loud. The code snippets come from the actual repository and are trimmed only for length.

Numbers worth memorising:

Fact	Value
Worked bill	Domestic, 3 kW, 30 days, 14208 → 14675 = **467 units, ₹2,575.70 energy, ₹150 fixed, ₹128.78 duty, ₹2,854.48 current demand**
Rollover example	Commercial, 99500 → 300 = **800 units**, not −99,200
Cleaning	30,000 raw rows → **28,021 kept, 1,979 rejected**, 8 rejection reasons
Exception classes	5: genuine_high_usage, estimated_read, meter_fault, tamper_suspected, tariff_mismatch
Tariff orders	**12** (4 financial years × 3 categories, 2 mid-year amendments)
Retrieval eval	**148** questions, 37 distinct question/category pairs
Assistant	**4 tools, 7 nodes**, 4 conditional exits
Tests	178 grader tests at Week 8 Day 3

Part 1: Objective, users and use cases

1. What is the objective of MeterMind?

MeterMind is a platform for an electricity distribution company that bills about twenty-five thousand low-tension connections across five divisions. The company had five specific problems, and the platform addresses each one.

First, clerks priced bills by hand and applied the slab table, fixed charge, duty and rebate in whatever order they'd learned, so the same reading could produce different bills. I fixed one pricing order in a single function that everything else calls.

Second, readings that couldn't physically have happened got argued over instead of refused. The platform now refuses them and states each reason in words.

Third, the vigilance team had about five thousand suspect readings and no way to rank them, and every genuine consumer wrongly sent to vigilance is a customer lost. I trained a classifier, chose its operating rule on cost rather than accuracy, and sorted readings into the five queues the company actually works.

Fourth, the grievance desk opened every complaint by hand. I built a pipeline that cleans, classifies, summarises and checks complaints, and escalates the ones a supervisor needs to see today.

Fifth, tariff wording is spread across twelve orders, some superseded, but the superseded ones still govern the cycles billed under them. My retrieval answers from the order that was in force for the billing period asked about.

On top of all that there's a consumer assistant behind one authenticated endpoint. It uses those earlier pieces as tools and stops for an inspector when it shouldn't answer alone.

2. What business problems existed, and what fixes each?

I think of it as a table that maps each pain point to a layer:

Problem	Where it's fixed
Clerks applied charges in different orders	`calculate_bill_amount` in `core/tariff_calculations.py` (Week 1)
Impossible readings were argued over	`refusal_reasons()` on the connection classes plus the cleaning rules (Weeks 1–2)
No way to rank 5,000 suspect readings	Champion model, cost-based operating rule, 5 queues (Weeks 3–5)
Every complaint opened by a person	Text cleaning, classifiers, attention summary, LLM tone, escalation (Weeks 5–6)
Superseded orders answered the wrong period	Date-filtered retrieval over ChromaDB (Weeks 6–7)
No self-service for consumers	LangGraph assistant with an inspector gate (Weeks 7–8)

What ties it together is that every layer reuses the one beneath it. The API, the screening endpoint and the assistant's `calculate_bill` tool all return what the Week 1 function returns, so there's only one definition of a bill in the whole system.

3. What is the business value, and how would you measure success?

Consistency is the first win: the same reading always produces the same bill, which removes a whole category of disputes. Beyond that, I'd measure it in the company's own terms.

- **Vigilance:** the hit rate of inspections, meaning what share of cases sent to vigilance turn out to be real tampering, and the number of genuine consumers wrongly flagged. That's exactly what my cost matrix encodes. Accuracy alone isn't the metric, because a model that predicts "genuine" for everything looks accurate on imbalanced data.
- **Grievance:** time to first response, and whether escalated complaints really were the urgent ones.
- **Retrieval:** on the 148-question evaluation set, whether the right order and section came back for the period asked about.
- **Assistant:** share of runs answered without a person, share held, and above all zero made-up figures, which the critic and gate are there to enforce.

4. Who are the end users?

There are three kinds of user, and the service enforces the difference.

The first is the **meter reader**, or the field application they use. They're anonymous: they submit a reading with no token and get back the priced bill plus any reasons it was refused. They can also read, replace or delete a reading by ID, check health, and sign up or log in.

The second is the **consumer**, with role `consumer`. With a token they can do everything open plus list the reading book with filters and paging, screen a reading against the model, and ask the assistant a question.

The third is the **inspector**, with role `inspector`. On top of the consumer's permissions, an inspector can move a case along the status ladder, which writes an audit row, and release an assistant answer the gate held.

Some people don't call the API at all: billing clerks use the clean book and the SQL reports, the vigilance team works the queue files and the exception report, and grievance supervisors use the escalation list.

Capability	Meter reader	Consumer	Inspector
`GET /health`, `POST /readings`, `GET/PUT/DELETE /readings/{id}`, sign up / login	✓	✓	✓
`GET /readings` (paged list), `POST /screening`, `POST /assistant/questions`		✓	✓
`PATCH /readings/{id}/status`, `POST /assistant/threads/{id}/release`			✓

5. Why can't a consumer move a case or release a held run?

Both actions have consequences for a real person. Moving a case to `billed_back` means recovering money from a consumer, and releasing a held answer means telling someone something about a reading the model thinks might be tampered with. A person has to make those decisions.

In code I have two FastAPI dependencies. `require_signed_in` only checks that the token can be read. `require_inspector` also checks the role, and that's where I separate 401 from 403:

```
def require_inspector(carried: HTTPAuthorizationCredentials = Depends(bearer_scheme)):
    token = None if carried is None else carried.credentials
    payload = read_token(token)                    # 401 if absent / expired / altered
    if payload.get("role") != constants.INSPECTOR_ROLE:
        raise HTTPException(status_code=403, detail="role inspector is required")
    return payload
```

A 401 means "I don't know who you are." A 403 means "I know who you are, and you aren't allowed to do this."

6. Why does a failed login give the same message for an unknown account and a wrong password?

So that nobody can use the login endpoint to find out which account numbers exist. If "no such account" and "wrong password" returned different messages, an attacker could enumerate valid accounts and then target them. Both cases come back as the same 401:

```
stored = accounts.get(account_number)
if stored is None or not verify_password(wanted.password, stored["hashed_password"]):
    raise HTTPException(status_code=401, detail="account number or password is wrong")
```

7. Walk me through a meter reader submitting a reading.

The meter reader sends `POST /api/v1/readings` with no token. Here's what happens in order.

First the request goes through the middleware stack: CORS, then a fixed-window rate limiter that allows 60 requests per minute per IP and returns 429 beyond that, then the request logger, which starts a timer.

Next, Pydantic validates the body against `ReadingSubmission`, so a malformed account number or a non-numeric load is rejected with 422 before my code runs.

Then the route hands the record to the factory, `create_service_connection`, which settles the category and returns the right subclass. An unsupported category becomes a 422 as well.

The connection computes its physical maximum once in the constructor: load × 24 × cycle days × the 1.10 allowance. From that it computes units, which handles rollover. Pricing then runs in the fixed order: energy, fixed charge, duty, rebate, and a clamp at zero. `refusal_reasons()` collects every rule the reading breaks.

The repository stores it and assigns an ID, I build the reply, and it goes back as 201. On the way out, the middleware appends a line like `1786648786.457|POST|/api/v1/readings|201|32.4` to `requests.log`.

```
@router.post("/readings", response_model=schemas.ReadingReply, status_code=201)
def submit_reading(submitted_reading: schemas.ReadingSubmission):
    record = _submission_as_record(submitted_reading)
    try:
        create_service_connection(record)
    except UnsupportedCategoryError as error:
        raise HTTPException(status_code=422, detail=str(error)) from error
    stored = reading_repository.add_reading(record)
    return price_reading(stored)          # units, energy, fixed, net, is_billable,
refusal_reasons
```

For the standard example (domestic, 3 kW, 30 days, 14208 → 14675) the reply carries 467 units and ₹2,854.48 with `is_billable: true`. If the meter had gone backwards with no rollover to explain it, `is_billable` would be false and `refusal_reasons` would say "the current reading is below the previous reading with no rollover".

I deliberately store refused readings and return them with reasons instead of throwing them away, because a refused reading is still evidence someone needs to look at.

8. Walk me through sign-up, login and a protected call.

Sign-up is `POST /accounts` with an account number, email, password and role. Pydantic validators check that the account number matches `ACC` followed by eight digits and that the email is well formed. I hash the password with Argon2, which is salted and deliberately slow, and only the hash is stored. A duplicate account number gets 409.

```
from argon2 import PasswordHasher
hasher = PasswordHasher()

def hash_password(plain_password):
    return hasher.hash(plain_password)          # salt is generated per hash
```

Login is `POST /accounts/login`. I verify against the stored hash and issue an HS256 JWT carrying the subject, the role and a 30-minute expiry. The signing secret comes from an environment variable, not from the code.

```
def issue_token(subject, role):
    expires = datetime.now(timezone.utc) + timedelta(minutes=constants.TOKEN_VALID_MINUTES)
    payload = {"sub": subject, "role": role, "exp": expires}
    return jwt.encode(payload, read_signing_secret(), algorithm="HS256")
```

For a protected call like `GET /readings?page=1&page_size=20`, the client sends `Authorization: Bearer <token>`. `require_signed_in` decodes it: a missing, expired, altered or foreign-signed token all return the same 401, "the token could not be read". If the token is good, the route filters, pages and prices the readings and returns `{readings, page, page_size, total}`.

9. Walk me through an inspector moving a case.

The inspector sends `PATCH /readings/RDG50000002/status` with something like `{"wanted_status": "assigned", "noted_by": "je/1180"}`.

`require_inspector` runs first and returns 401 or 403 if needed. Then I load the reading, which can give 404. Then I ask the status ladder whether the move is legal. The ladder is a plain dictionary of allowed transitions:

```
ALLOWED_TRANSITIONS = {
    "raised":          ("assigned",),
    "assigned":        ("inspected",),
    "inspected":       ("regularised", "billed_back", "closed_no_fault", "assigned"),
    "regularised":     (),
    "billed_back":     (),
    "closed_no_fault": (),
}
```

An illegal move raises a domain error that the route turns into 409, with a message such as "a case at raised may move to assigned, not to closed_no_fault". A legal move is saved, and one row is appended to `output/audit_trail.csv`:

```
recorded_at,actor,role,reading_id,from_status,to_status
2026-08-13T19:19:46.541715+00:00,je/1180,inspector,RDG50000002,raised,assigned
```

Two design points. The three outcome states are terminal, so a closed case can't be quietly reopened. And `inspected → assigned` exists so an inconclusive inspection can be sent back out instead of being forced to a verdict.

10. Walk me through screening a reading.

A signed-in user sends `POST /screening` with a reading's fields. I validate the category through the factory as usual, then load the champion and its operating rule once and cache them. If no champion has been trained yet, that's a 503 with a clear message rather than a crash.

```
model, rule = load_champion_once()
widened = widen_readings(pd.DataFrame([record]))              # same 3 derived features
columns = widened[list(modelling_columns(True))]
```

```
decided = predict_with_rule(model, columns, rule["tamper_floor"], rule["fault_floor"])[0]
confidence = float(model.predict_proba(columns)[0][_class_labels(model).index(decided)])
return schemas.ScreeningReply(exception_class=decided, confidence=round(confidence, 4),
                              goes_to=Path(constants.QUEUE_FILE_PATHS[decided]).name, ...)
```

The important part is that the saved model is a full scikit-learn `Pipeline`, preparation included, and the request goes through the same `widen_readings` function used in training. That means there's no training/serving skew: the endpoint can't prepare a reading differently from how the model was trained. The reply tells you the class, how confident the model is, and which queue file it goes to, for example `queue_vigilance.csv`.

11. Walk me through a consumer asking the assistant.

The consumer sends `POST /assistant/questions` with a question, their consumer category and a billing period. If they don't pass a thread ID, I generate a UUID. Then I invoke a LangGraph graph that was compiled with a SQLite checkpointer.

1. **Triage** runs my Week 5 complaint classifiers on the question to get a category and urgency. An empty question is refused here, and the router ends the run.
2. **Plan**: an LLM is given a tool catalogue built automatically from the tools' own Pydantic schemas, and must return JSON with a list of `{tool, arguments}` steps, at most three.
3. **Tools** checks every name in the plan against the registry first. If any name is unknown, the whole plan is refused and nothing runs. Otherwise the tools run in the planned order, and each failure is recorded as a result instead of crashing the run.
4. **Draft**: an LLM writes the reply only from what the tools returned.
5. **Critique**: a second LLM call can only veto. It can't rewrite.
6. **Gate** holds the run if a tool returned `tamper_suspected` or `tariff_mismatch`, if the critic vetoed, or if nothing was drafted.
7. **Respond** turns the draft into plain language for the consumer.

The response has `thread_id, answer, held, refusal, category, urgency, steps, tools_called`.

There are three outcomes I can demonstrate. RDG20000011 is a genuine high-usage domestic cycle, so the tools answer, the draft carries real figures, it passes the gate, and `steps` ends with `respond`. RDG20000034 is read as `tamper_suspected`, so the consumer gets the standard "this has gone to an inspector" reply, `held` names the reason, and `steps` stops at `gate`. And if the question names no reading at all, the lookup fails, the draft says plainly that the reading isn't available, and no rupee figure is made up. If Gemini is down, the route returns 503.

12. Walk me through an inspector releasing a held answer.

The inspector sends `POST /assistant/threads/{thread_id}/release`. The graph is compiled with `interrupt_before=["respond"]`, so every run pauses just before responding. For an ordinary run, my workflow continues it straight away. A held run stays paused, and its full state is saved in `database/assistant_threads.sqlite` under that thread ID.

To release it, I check the thread really is waiting at `respond` (otherwise 404), clear `held` as if the gate itself had written it, and resume:

```
def release_held_run(thread_id, graph=None):
    snapshot = graph.get_state(_config(thread_id))
    if "respond" not in tuple(snapshot.next or ()):
        raise NoSuchThread(f"thread {thread_id} is not waiting")       # -> 404
    graph.update_state(_config(thread_id), {"held": ""}, as_node="gate")
    graph.invoke(None, _config(thread_id))                             # runs respond only
    return _outcome(thread_id, graph.get_state(_config(thread_id)).values)
```

Nothing is recomputed: the tools aren't re-run, and the draft the critic approved is the one that gets phrased. That's what makes this genuine human-in-the-loop instead of "ask again". And because the state lives in SQLite, the release works even if the server restarted in between.

13. How does a tariff question reach the right order when there are 12?

I don't leave that decision to vector similarity. The twelve orders say very similar things in very similar words, so the nearest chunk is quite often from the wrong year.

So the date logic happens first, deterministically, against the manifest. I find the one order whose window contains the first day of the billing period for that category. The manifest tiles the dates with no gaps or overlaps, so exactly one matches:

```
def order_in_force(manifest, consumer_category, billing_period):
    start = _period_start(billing_period)                       # 2025-05 -> 2025-05-01
    for order in manifest:
        if not _category_matches(order.get("consumer_category"), consumer_category):
            continue
        if effective_from and start < effective_from: continue
        if effective_to   and start > effective_to:   continue
        return order
    raise NoOrderInForce(f"no {consumer_category} order was in force in {billing_period}; ...")
```

Then I pass that as a Chroma `where` filter, so the similarity search runs only inside that one order:

```
conditions = {"order_id": order["order_id"], "consumer_category": order["consumer_category"]}
search_with_filter(collection, question, embedder, conditions)   # -> {"$and": [...]}
```

The same question about domestic fixed charges for 2024-09 and for 2026-06 reaches two different orders. And there's a test proving that without the filter an out-of-force extract does come back, which is why the filter exists. A period with no order in force raises `NoOrderInForce` instead of quietly answering from the nearest year.

14. What runs in batch, outside the API?

There are four back-office pipelines.

1. **Cleaning (Week 2):** the raw reading book becomes `readings_clean.csv` plus `readings_rejected.csv`, which keeps a reason column. The clean book is loaded into SQLite, and three SQL reports come out of it: exception rate by category, estimated-read share by month, and billed units by division.
2. **Queues (Week 4 Day 5):** `reading_book_predict.csv` goes through the same feature widening, then the champion and its operating rule, and comes out as five queue files: no action, meter reading, metering, vigilance, billing. The Week 5 exception report adds per-class precision and recall, which the assistant later quotes.
3. **Complaints (Weeks 5-6):** clean the text, classify category and urgency, summarise threads with attention, read tone through Gemini, extract account numbers and meter serials and check them against the register, then decide escalation.
4. **Indexing (Weeks 6-7):** read the manifest, extract PDF text, chunk it, embed it with all-MiniLM-L6-v2, and store it in a persistent Chroma collection.

The batch runner never lets one bad message stop the run. Each failure becomes a row in `batch_failures.csv` and the loop carries on:

```
for position, message in enumerate(messages):
    try:
        results.append(handle_one(message))
```

```
    except MessageFailure as failure:
        results.append(failure)
    except Exception as error:
        results.append(MessageFailure(_message_id(message, position), str(error)))
```

15. Trace one reading through the whole platform.

I'll follow one row all the way through.

1. It starts as a row in the raw CSV, possibly with a date in one of four shapes, like `13-Jun-2025`.
2. Week 2 cleaning parses the date against the four known formats and checks the eight rules. It either lands in the clean book or in the rejected file with a reason.
3. Clean rows are loaded into SQLite, where the reports and the assistant's `look_up_reading` tool can find them.
4. The same reading can be submitted through `POST /readings`, where the Week 1 domain prices it and lists any refusal reasons.
5. `POST /screening` runs it through the champion. Say it comes back `tamper_suspected`, so it's routed to `queue_vigilance.csv`.
6. The consumer, worried about their bill, asks the assistant. The assistant looks up the reading, prices it with the Week 1 function, gets the class from the champion, and pulls the relevant section from the tariff order in force for that period.
7. Because the class is `tamper_suspected`, the gate holds the run. The consumer gets the holding message, and the state is saved under the thread ID.
8. An inspector moves the case `raised → assigned → inspected`. Each move is checked by the ladder and written to the audit trail.
9. The inspector calls release. The approved draft is phrased and sent to the consumer.
10. Every HTTP call along the way, including the 401s, 404s and 409s, is a line in `requests.log`.

That's diagram 22 in `docs/diagrams`.

Part 2: Architecture

16. Walk me through MeterMind end to end.

I built it over eight weeks, one layer at a time, and each layer only depends on the ones below it.

- **Week 1** is the billing domain in plain Python: units, slab tables, fixed charge, duty, rebate, then an exception hierarchy, a connection class hierarchy with a factory, validators, decorators and proration.
- **Week 2** cleans the reading book, loads it into SQLite, and exposes the domain through FastAPI.
- **Week 3** adds the SQLAlchemy repository, middleware, Argon2 password hashing, JWT and role checks, and the case status ladder.
- **Weeks 3 to 5** are the ML side: a shared preparation pipeline, four model families, cross-validation, three derived features, a champion with a cost-based operating rule, five queues, and a screening endpoint.
- **Weeks 5 and 6** handle complaints: cleaning, classifiers, attention summaries, a Gemini client, reference extraction, batch processing and escalation.
- **Weeks 6 and 7** are retrieval over the twelve tariff orders, with grounded, cited answers filtered to the order in force.
- **Weeks 7 and 8** wrap four of those earlier pieces as tools and drive them with a LangGraph assistant behind an authenticated endpoint, with a human-in-the-loop gate.

17. Why build it in layers, and what depends on what?

Each layer is built on the one before it, and I kept the dependency arrows pointing one way. `core` imports nothing from the platform except config. `data_access` uses `core`. The API uses `core` and `data_access`. The models use the clean data. The assistant uses everything as tools. Nothing in `core` knows FastAPI exists.

The practical benefit is a single source of truth. There's exactly one function that prices a bill:

```
def calculate_bill_amount(consumer_category, units, sanctioned_load_kw):
    energy_charge = calculate_slab_energy_charge(consumer_category, units)
    fixed_charge = apply_fixed_charge(consumer_category, sanctioned_load_kw)
    electricity_duty = apply_electricity_duty(consumer_category, energy_charge)
    rebate = apply_subsidy(consumer_category, units)
    amount = energy_charge + fixed_charge + electricity_duty - rebate
    if amount < 0:
        amount = 0.0
    return amount
```

The assistant's `calculate_bill` tool is a thin wrapper around that function. If the tariff changes, I change it in one place and the API, the batch jobs and the assistant all agree.

All the published figures (slab tables, limits, ID patterns, file paths) live in `config/constants.py`. The rule I followed is that constants hold what the company *publishes*, and code holds what has to be *derived*.

18. What is the core rule separating tools from agents?

Tools compute, and agents only choose and phrase. An LLM never does arithmetic and never states a figure a tool didn't return.

I enforce that at several points, not just in one prompt.

- The **planner** may only name tools from a catalogue I generate from the tools' Pydantic schemas, and `run_tools` refuses the whole plan if it names anything unknown.
- The tools validate their own arguments. `calculate_bill` has `sanctioned_load_kw: float = Field(gt=0)`, so a made-up zero load is refused by the schema, not by the model's judgement.
- The **resolver** prompt says to draft only from what the tools returned and to say plainly where they didn't answer.
- The **critic** may only veto. It can't rewrite, so it can't introduce numbers of its own.
- The **gate** holds the run if the critic objected.

```
CRITIC_PROMPT = (
    "You may only object. You may not rewrite the draft and you may not answer "
    "the question itself.\n"
    "Object where the draft carries a figure the tools did not return, a "
    "statement contradicting what they did return, or an accusation against the "
    "consumer that the tools did not establish.\n" ...
)
```

The reason is auditability. When a consumer disputes a bill, I can point to the exact function and inputs that produced every rupee in the reply.

Part 3: Billing domain (Week 1)

19. Telescopic vs non-telescopic slabs: how did you do both?

Telescopic means each block is charged at its own rate, and only the units that fall inside a block pay that block's rate. Domestic works that way. For 467 units:

Block	Units	Rate	Charge
0–100	100	₹3.50	350.00
101–200	100	₹4.90	490.00

Block	Units	Rate	Charge
201–400	200	₹6.30	1,260.00
401+	67	₹7.10	475.70
			2,575.70

Non-telescopic, which is commercial, charges the whole consumption at the rate of the block it lands in. So one unit past a boundary reprices the entire cycle. That's intended, and I wrote a test for exactly that cliff.

Which convention applies is decided by a published tuple, never by writing a category name into the arithmetic:

```
TELESCOPIC_CATEGORIES = ("domestic", "agricultural")

def calculate_slab_energy_charge(consumer_category, units, blocks=None):
    if blocks is None:
        blocks = SLAB_TABLE_INR_BY_CATEGORY[consumer_category]

    if consumer_category not in TELESCOPIC_CATEGORIES:          # non-telescopic
        for limit, rate in blocks:
            if limit is None or units <= limit:
                return units * rate
        return units * blocks[-1][1]

    energy_charge, remaining_units, previous_limit = 0.0, units, 0   # telescopic
    for limit, rate in blocks:
        if remaining_units <= 0:
            break
        slab_units = min(remaining_units, limit - previous_limit)
        energy_charge += slab_units * rate
        remaining_units -= slab_units
        previous_limit = limit
    return energy_charge
```

Agricultural has a single block, so it comes out the same either way, and there's a test for that too. The optional `blocks` argument exists because Week 1 Day 5 passes in prorated tables when a tariff changes mid-cycle.

20. How do you tell a meter rollover from a meter going backwards?

A five-digit meter rolls over at 100,000. If the current reading is below the previous one, either the meter wrapped past 99,999 or it went backwards, which is a fault or tampering. Only a rollover is billable.

My test is physical: could this connection have drawn enough to reach the ceiling? I credit a rollover only if the previous reading plus the physical maximum reaches 100,000, and even then the units across the boundary must not exceed what the connection could draw.

```
def calculate_units_consumed(previous_reading, current_reading, physical_maximum_units):
    if current_reading >= previous_reading:
        return current_reading - previous_reading
    if previous_reading + physical_maximum_units < METER_ROLLOVER_UNITS:
        return 0.0                                  # went backwards: nothing billable
    rollover_units = current_reading + METER_ROLLOVER_UNITS - previous_reading
    if rollover_units <= physical_maximum_units:
        return rollover_units
    return 0.0
```

So a 30 kW commercial meter going from 99,500 to 300 is 500 + 300 = 800 units, not −99,200. A domestic meter going from 14,675 back to 14,208 returns zero units, and the refusal reason says in plain words that the current reading is below the previous one with no rollover to explain it.

21. How do you compute the physical maximum, and why only once?

It's sanctioned load in kW × 24 hours × cycle days × a 1.10 allowance, because meters are allowed a little over their sanctioned load. I compute it once in the connection's constructor:

```
self.physical_maximum_units = (
    sanctioned_load_kw * HOURS_IN_A_DAY * cycle_days * CAPACITY_ALLOWANCE_FACTOR
)
self.units_consumed = calculate_units_consumed(
    previous_reading, current_reading, self.physical_maximum_units
)
```

The reason for once is that two separate decisions depend on it: whether a rollover is credited, and whether consumption is impossible. If they computed it separately, someone could change one formula and not the other, and then a reading could be credited as a rollover and also refused for impossible consumption.

There's also a subtle second threshold. In cleaning I use 2.5× instead of 1.10×. A reading a little over the sanctioned load is *suspicious*: the load on file is probably wrong, which is what `tariff_mismatch` means, so it should be flagged for the model. A reading far beyond any load is *corrupt*, and that row is refused outright.

22. Why does the order of charges matter?

Because the steps depend on each other. Duty is a percentage of the energy charge alone, not of energy plus fixed, so the energy charge has to exist first. The rebate is means-tested on units. The bill is clamped at zero last.

```
units → energy charge → fixed charge → duty (% of energy only) → minus rebate → max(0, ·)
```

For the example: energy ₹2,575.70, plus fixed 3 kW × ₹50 = ₹150, plus duty at 5% of 2,575.70 = ₹128.78, minus rebate ₹0 (467 is above the 100-unit threshold) = **₹2,854.48**.

If someone applied duty to the whole bill, it would come to ₹136.29 instead of ₹128.78. That's exactly the clerk-to-clerk inconsistency the project was asked to remove.

The rebate is a hard cliff, not a taper: 100 units earns ₹1.50 on every unit, and 101 units earns nothing.

```
def apply_subsidy(consumer_category, units):
    if units <= SUBSIDY_QUALIFYING_UNITS_BY_CATEGORY[consumer_category]:
        return units * SUBSIDY_INR_PER_UNIT_BY_CATEGORY[consumer_category]
    return 0.0
```

The clamp at zero matters for agricultural connections, whose rebate of ₹2.60 a unit is larger than their ₹2.25 tariff. Without it, a small farm cycle would produce a negative bill.

23. Why is agricultural billed per horsepower?

Because the tariff publishes it that way, since pumps are rated in horsepower. I convert kW to HP and then apply the HP rate:

```
def apply_fixed_charge(consumer_category, sanctioned_load_kw):
    if consumer_category == "agricultural":
        return sanctioned_load_kw * HORSEPOWER_PER_KILOWATT * FIXED_CHARGE_INR_PER_HORSEPOWER
    return sanctioned_load_kw * FIXED_CHARGE_INR_PER_KW_BY_CATEGORY[consumer_category]
```

For 3 kW that's 3 × 1.34102 × ₹22 ≈ ₹88.51. If you mistakenly billed by the kW, there isn't even a kW rate for agricultural in the schedule, and you'd produce a different bill, not a rounding difference.

24. Explain the class hierarchy and the factory.

`ServiceConnection` is both the domestic connection and the base class. It holds the six fields, computes the physical maximum and units once, and has methods for every charge plus `refusal_reasons()` and `is_billable()`. The subclasses override only what their category bills differently:

```
class CommercialConnection(ServiceConnection):
    def energy_charge_inr(self):                    # non-telescopic
        for upper, rate in self.applicable_slab_table():
            if self.units_consumed <= upper:
                return self.units_consumed * rate
        return 0.0

class AgriculturalConnection(ServiceConnection):
    def fixed_charge_inr(self):                     # per horsepower
        return self.sanctioned_load_kw * HORSEPOWER_PER_KILOWATT *
FIXED_CHARGE_INR_PER_HORSEPOWER
```

Callers never decide which class to build. They hand a record to the factory, which also tidies up the input:

```
def create_service_connection(record):
    category = str(record["consumer_category"]).strip().lower()
    classes = {"domestic": ServiceConnection,
               "commercial": CommercialConnection,
               "agricultural": AgriculturalConnection}
    if category not in classes:
        raise UnsupportedCategoryError(category)
    return classes[category](str(record["account_number"]).strip().upper(), category, ...)
```

That's polymorphism plus the open/closed principle. Adding a category means adding a subclass and one dictionary entry, and the routes, batch jobs and tools don't change.

25. Why a custom exception hierarchy?

So a caller can catch one parent and be certain nothing else the platform raises on purpose will escape.

```
class MeterMindError(Exception): pass
class UnsupportedCategoryError(MeterMindError): pass
class ReadingStorageError(MeterMindError): pass
class InvalidReadingError(MeterMindError): pass
class ToolError(MeterMindError): pass
```

It also gives the API a clean mapping from domain errors to HTTP codes: `UnsupportedCategoryError` → 422, `ReadingStorageError` → 404, an illegal case move → 409. In the assistant, `ToolError` means "this tool couldn't answer". The tools node catches it and records it as a result, so the draft can say "that reading isn't on file" instead of the whole run crashing.

Related: I wrote decorators like `@reject_negative_readings`, which raises `InvalidReadingError` for any negative numeric argument, and `@measure_execution_duration`, which records timing on the wrapper. Both use `functools.wraps` so the function keeps its name and docstring.

Part 4: Data and persistence (Weeks 2–3)

26. What were the cleaning rules, and why reject with a reason?

There are eight rules, checked in order, and the first one a row breaks becomes its rejection reason:

```
REJECTION_REASONS = (
    "a required field is empty",
    "the read date could not be read",
    "the billing period is not a year and a month",
    "the consumer category has no tariff schedule",          # temp, misc, streetlight, industial
    "the sanctioned load is not above zero",
    "the billing cycle length is outside the permitted range",    # 26–36 days
    "the current reading is below the previous reading",        # and no rollover explains it
    "the consumption could not be drawn by any load on file",
)
```

On the training book, 28,021 rows survive and 1,979 are rejected.

I reject with a reason instead of silently dropping rows for three reasons. It's auditable: anyone can open `readings_rejected.csv` and see why each row went. It tells the field team what to fix at the source. And it protects the model: if I'd quietly dropped rows, I'd never know whether a rule was too aggressive and was throwing away the very readings that are interesting, such as tampering.

27. How did you parse dates in four formats?

Explicitly, one format at a time, and never re-parsing a row that already succeeded. I deliberately didn't use pandas' lenient inference, because `03/04/2025` silently becomes March 4th instead of April 3rd, and that's a wrong date, not a missing one.

```
READ_DATE_FORMATS = ("%Y-%m-%d", "%d/%m/%Y", "%d-%b-%Y", "%d.%m.%Y")

mask = result.isna()
result = result.mask(mask, pd.to_datetime(text.where(mask), format="%Y-%m-%d", errors="coerce"))
mask = result.isna()
result = result.mask(mask, pd.to_datetime(text.where(mask), format="%d/%m/%Y", errors="coerce"))
# ... "%d-%b-%Y" is parsed via an explicit English month map so it doesn't depend on locale
# ... then "%d.%m.%Y"
```

It's vectorised, so it runs on the whole column at once, and anything still `NaT` at the end becomes "the read date could not be read". For the `13-Jun-2025` format I map month names through my own dictionary so the result doesn't change with the machine's locale.

28. Why a repository interface with in-memory and SQL versions?

The routes depend on an abstract `ReadingRepository`, not on any particular storage:

```
class ReadingRepository(abc.ABC):
    @abc.abstractmethod
    def add_reading(self, reading): ...
    @abc.abstractmethod
    def get_reading(self, reading_identifier): ...
    @abc.abstractmethod
    def replace_reading(self, reading_identifier, reading): ...
    @abc.abstractmethod
```

```
    def list_readings(self): ...
    @abc.abstractmethod
    def remove_reading(self, reading_identifier): ...
```

There are two implementations. `InMemoryReadingRepository` is a dictionary that returns deep copies, so a caller can't change stored state by accident. `SqlReadingRepository` uses SQLAlchemy and opens and closes a session per call in `try/finally`. Both raise `ReadingStorageError` for a missing ID, so the route maps it to 404 the same way regardless of storage.

The application factory chooses between them:

```
def choose_reading_repository(database_path):
    if not database_path:
        return InMemoryReadingRepository()
    return SqlReadingRepository(database_path)
```

Tests use the in-memory version and are fast and isolated. `python -m metermind.api.application_factory` runs on SQLite. That's dependency inversion in practice.

I also keep two SQLite files on purpose. The reporting database is rebuilt from the clean book every time, while the service's ORM database persists. A service whose rows vanish when someone reloads the reports isn't really a store.

Part 5: API and security (Weeks 2–3)

29. How is authentication done?

Passwords are hashed with Argon2, which is salted, slow and memory-hard, the current recommendation. Login verifies against the hash and returns a JWT signed with HS256 that carries `sub`, `role` and a 30-minute `exp`. The signing secret comes from the `METERMIND_TOKEN_SECRET` environment variable. Protected routes declare `Depends(require_signed_in)` or `Depends(require_inspector)`, so authorisation is visible in each route's signature instead of buried in its body.

`verify_password` catches every Argon2 failure, including mismatches and malformed hashes, and returns `False`, so a corrupted hash can't turn into a 500 that tells an attacker something.

30. How does role-based access work, and what's the case ladder?

There are three levels: open, signed in, and inspector, enforced by the two dependencies I showed earlier. The case ladder is `raised → assigned → inspected → {regularised | billed_back | closed_no_fault}`, with `inspected → assigned` allowed for a revisit. The three outcomes are terminal. An illegal move is a 409, and every legal move writes an audit row recording who, in what role, which reading, from and to.

Keeping the ladder as a plain dictionary in its own module (`case_transitions.py`) means business rules can be reviewed and tested without touching any HTTP code.

31. What does your middleware do?

It does three things, added in the application factory.

The **request logger** writes one pipe-separated line per request with the timestamp, method, path, status and milliseconds:

```
async def dispatch(self, request, call_next):
    started = monotonic()
    received_at = time()
```

```
    response = await call_next(request)
    duration_ms = round((monotonic() - started) * 1000, 3)
    line = "|".join(str(v) for v in (received_at, request.method, request.url.path,
                                     response.status_code, duration_ms))
    with self.log_path.open("a", encoding="utf-8") as handle:
        handle.write(line + "\n")
    return response
```

I use `monotonic()` for the duration because wall-clock time can jump.

The **rate limiter** is a fixed window, 60 requests per 60 seconds per client IP, returning 429. It's simple, but enough to stop someone brute-forcing the login.

CORS only allows the two published origins.

I log failures too (401, 404, 409, 503) because a log that only records what worked is useless on the day something breaks. A burst of 401s is exactly what I'd want to see when someone is guessing passwords.

32. Where does each status code show up?

Code	When
200 / 201 / 204	Success; 201 for a new reading or account; 204 for a delete
401	No token, or the token is expired, altered or signed by someone else; also a failed login
403	Valid token, wrong role, e.g. a consumer trying to move a case
404	Unknown reading, or releasing a thread that isn't waiting
409	Illegal case move, or signing up an account number that already exists
422	Body fails Pydantic validation, or the category has no schedule
429	Over the rate limit
503	No champion trained yet for screening, or the language model is unreachable

I'm careful about 401 vs 403 in particular, and about not telling the client more than it needs to know.

Part 6: Machine learning (Weeks 3–5)

33. Why share one preparation pipeline across all models?

For two reasons: no leakage and a fair comparison. I built a `ColumnTransformer` once. Numbers get a median imputer and a standard scaler. Categories get a most-frequent imputer and a one-hot encoder with `handle_unknown="ignore"` and `min_frequency=0.01`, so rare levels are grouped together and an unseen category at prediction time doesn't crash.

```
def build_feature_pipeline(widened=False):
    number_columns = list(MODELLING_NUMBER_COLUMNS) + (list(DERIVED_NUMBER_COLUMNS) if widened
else [])
    number_steps = Pipeline([("fill", SimpleImputer(strategy="median")), ("scale",
StandardScaler())])
    category_steps = Pipeline([("fill", SimpleImputer(strategy="most_frequent")), ("widen",
_one_hot_encoder())])
    return ColumnTransformer([("numbers", number_steps, number_columns),
                              ("categories", category_steps, list(MODELLING_CATEGORY_COLUMNS))])
```

Every model is `Pipeline([("prepare", build_feature_pipeline()), ("decide", <estimator>)])`. Because the preparation sits inside the pipeline, cross-validation refits the imputer and scaler on each training fold only, so the validation fold never leaks into the medians or means. And the saved `.joblib` file carries its preparation with it, which is why the screening endpoint can't drift from training.

Everything uses seed 42 and the same 80/20 stratified split, so any two models are directly comparable.

34. How did you choose the champion?

I compared four families: logistic regression as the baseline, a decision tree with depth limited to 8, a random forest with 200 trees and depth 14, and a linear SVM. I used five-fold `StratifiedKFold` with shuffling and a fixed seed, so every fold has the same class mix and the folds are the same for every model.

```
splitter = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for name, model in candidates.items():
    scores = cross_val_score(model, fitting_columns, fitting_classes, cv=splitter)
```

I look at the mean and also the spread across folds. A model that's best on average but swings widely is a riskier choice.

One deliberate decision: the SVM isn't eligible to be champion, because `LinearSVC` doesn't produce probabilities, and my operating rule needs a confidence per class. So the champion is the best mean among logistic regression, tree and forest.

Why cross-validation instead of one split? A single 80/20 split can be lucky or unlucky, especially for the rare classes like tampering. Five folds give a mean and a variance, and a decision based on both is more defensible.

I also compared `class_weight=None` against `"balanced"` for the trees, because the classes are imbalanced.

35. What three features did you derive?

These are signals that are implied by the book but not written in it.

1. **`consumption_ratio_to_average`**: this cycle's units ÷ the twelve-month average. A sudden spike, or a sudden collapse, which is a classic tampering signature, shows up directly. If the average is missing or zero I set it to 1.0, meaning "normal", instead of dividing by zero.
2. **`load_factor`**: units ÷ (sanctioned kW × 24 × cycle days). How hard the connection is being used relative to its sanction. A load factor near or above 1 means the load on file is probably wrong, which is `tariff_mismatch`.
3. **`meter_age_years`**: from the install date to the end of the cycle. Older meters fail more, so it helps with `meter_fault`.

```
def add_load_factor(book):
    capacity = load * constants.HOURS_IN_A_DAY * cycle_days
    factor = units / capacity
    factor = factor.mask(~np.isfinite(factor))     # zero capacity -> NaN, imputed later, never
±inf
    result["load_factor"] = factor
```

I measured the benefit honestly: the same random forest on narrow vs widened columns, same split, with the difference written to `feature_lift.json`. The features are only kept because they earn their place.

36. Why choose the operating rule on cost and not accuracy?

Because the mistakes don't cost the same, and the data is imbalanced. Calling a genuine high-usage consumer a tamperer means sending vigilance to an honest customer's door, so the company loses that customer. Missing a real tamper loses revenue, but it's recoverable. So I wrote that down as a 5×5 cost matrix, with rows as the truth and columns as the prediction:

```
COST_MATRIX = {
    "genuine_high_usage": {"genuine_high_usage": 0.0, "estimated_read": 1.0, "meter_fault": 1.5,
                           "tamper_suspected": 25.0, "tariff_mismatch": 2.0},
    "tamper_suspected":   {"genuine_high_usage": 4.0, ...,  "tamper_suspected": 0.0, ...},
```

```
    ...
}
```

Genuine predicted as tamper costs 25. Tamper predicted as genuine costs 4. That asymmetry is the business decision.

The operating rule is a pair of confidence floors. The model may only name `tamper_suspected` if its probability is at least the tamper floor, and similarly for `meter_fault`. Otherwise it falls back to its second choice. I grid-search the floors on the held-out set and keep the pair with the lowest expected cost:

```
def predict_with_rule(model, columns, tamper_floor, fault_floor):
    floors = {"tamper_suspected": tamper_floor, "meter_fault": fault_floor}
    probabilities = model.predict_proba(columns)
    for row in probabilities:
        ranked = list(np.argsort(row)[::-1])
        chosen = labels[ranked[0]]
        if chosen in floors and row[ranked[0]] < floors[chosen]:
            chosen = labels[ranked[1]]          # not confident enough -> second choice
        ...

for tamper_floor in (0.0, 0.4, 0.5, 0.6, 0.7, 0.8):
    for fault_floor in (0.0, 0.4, 0.5, 0.6, 0.7, 0.8):
        cost = expected_cost(held_out_classes, predict_with_rule(model, X, tamper_floor,
fault_floor))
```

Floors of 0.0 reproduce the plain argmax exactly, so the rule can only match or beat the unaided model on cost. The chosen floors and their cost are saved to `operating_rule.json` next to the champion, and every consumer of the model (queues, screening, the assistant tool) applies the same rule.

37. How are flagged readings sorted into queues?

Each of the five exception classes maps to a work queue the company already runs:

```
QUEUE_FILE_PATHS = {"genuine_high_usage": "output/queue_no_action.csv",
                    "estimated_read":     "output/queue_meter_reading.csv",
                    "meter_fault":        "output/queue_metering.csv",
                    "tamper_suspected":   "output/queue_vigilance.csv",
                    "tariff_mismatch":    "output/queue_billing.csv"}
```

I take `reading_book_predict.csv` (IDs from RDG30000000, no labels), widen it with the same three features, apply the champion with its operating rule, and write each class's rows to its queue file. An empty queue is still written with a header, so downstream consumers never see a missing file.

The value of the cost-based rule shows up here. The vigilance queue is smaller and more precise than an argmax model would make it, so the team's limited inspection capacity goes to the cases most likely to be real. The Week 5 exception report then publishes per-class precision and recall, and the assistant quotes that precision whenever it mentions a class.

Part 7: NLP and complaints (Weeks 5–6)

38. How did you clean complaint text, and why does it matter?

Real complaints arrive wrapped in email noise: a quoted earlier reply, "Dear Sir", "Regards", a phone number and staff ID in the signature. If I left that in, the classifier would learn that "Dear Sir" predicts some category, which is learning the envelope instead of the letter.

I wrote a set of regexes, compiled once, and applied them in order: quoted reply, greeting and sign-off, signature, complaint reference numbers, then whitespace.

```
def clean_message(message):
    text = _as_text(message)
    cleaned = strip_quoted_reply(text)              # "> ...", "On ... wrote:", "--- Original
Message ---"
    cleaned = strip_greeting_and_sign_off(cleaned)
    cleaned = strip_signature(cleaned)              # phone numbers, STF-1234
    cleaned = COMPLAINT_REFERENCE_PATTERN.sub(" ", cleaned)
    cleaned = normalise_whitespace(cleaned)
    if cleaned:
        return cleaned
    # A cleaning that empties a message has removed the complaint itself.
    fallback = normalise_whitespace(COMPLAINT_REFERENCE_PATTERN.sub(" ", text))
    return fallback or normalise_whitespace(text)
```

That fallback is a safety net I care about: cleaning should never produce an empty message. If it would, I keep more instead of less.

The classifiers are TF-IDF on word unigrams and bigrams with `sublinear_tf` and `min_df=3`, feeding logistic regression. I put the cleaning inside the pipeline as a `FunctionTransformer`, so the saved model cleans raw text itself and no caller can forget to:

```
Pipeline([("clean", FunctionTransformer(clean_messages, validate=False)),
          ("weigh", TfidfVectorizer(ngram_range=(1, 2), min_df=3, max_features=40000,
sublinear_tf=True)),
          ("decide", LogisticRegression(max_iter=2000, random_state=42))])
```

There's one model for category (high_bill, no_supply, meter_problem, connection_request, payment_issue) and one for urgency (routine, priority, emergency). Each is tested against the trivial baseline of always predicting the most common label, and `classify_message` refuses to return a label outside the published sets.

39. Explain the attention-based thread summary.

I implemented scaled dot-product attention from scratch in NumPy, the same formula as in transformers: softmax($QK^T/\sqrt{d}$). I used the numerically stable softmax that subtracts the row maximum first.

```
def softmax_rows(scores):
    shifted = scores - np.max(scores, axis=-1, keepdims=True)
    exponentiated = np.exp(shifted)
    return exponentiated / np.sum(exponentiated, axis=-1, keepdims=True)

def attention_weights(queries, keys):
    scores = queries @ np.swapaxes(keys, -1, -2) / np.sqrt(keys.shape[-1])
    return softmax_rows(scores)
```

For a thread, I join its turns in order, clean each one, split into sentences, and drop fragments under 25 characters. Each sentence becomes a bag-of-words vector, and I run self-attention with the sentence matrix as both queries and keys. A sentence's score is how much attention it receives from the *other* sentences, so I subtract the diagonal, because every sentence attends most to itself.

```
def score_sentences(sentences):
    matrix = sentence_matrix(sentences)
```

```
    weights = attention_weights(matrix, matrix)
    return weights.sum(axis=0) - np.diag(weights)
```

I take the top three, then put them back in their original order so the summary reads naturally.

The intuition is that the sentence the rest of the thread keeps referring back to is the one that matters, which is usually the actual problem. Threads too short to compare get a reason written to the "not summarised because" column instead of a fake summary.

Then the summary or message goes to Gemini for something counting can't give me: tone (calm, frustrated or angry), returned as JSON with a quoted phrase as evidence.

40. How do you extract and check account numbers and meter serials?

The patterns are the published formats from constants: `ACC` + 8 digits, `MTR-XX-` + 6 digits, `CMP` + 7 digits. I wrap them in lookarounds so they only match whole tokens. For example, `ACC123456789` with nine digits must not yield a valid-looking eight-digit account.

```
def _whole_reference(published):
    body = published.lstrip("^").rstrip("$")
    return re.compile(rf"(?<![A-Za-z0-9-]){body}(?![A-Za-z0-9-])")
```

Then I check against `connection_register.csv` for two kinds of problem. An account the consumer quotes might not exist. Or the consumer quotes a meter serial that is registered to a *different* account than the one they quoted, which is either a typo or something worth a closer look.

```
def check_against_connection_register(accounts, serials, known_accounts, serial_owner):
    unknown_accounts = sorted({a for a in accounts if a not in known_accounts})
    on_another = [s for s in sorted(set(serials))
                  if serial_owner.get(s) is not None and serial_owner[s] not in set(accounts)]
    return {"unknown_accounts": unknown_accounts, "serials_on_another_account": on_another}
```

The point is that nothing a consumer types gets trusted just because it matches the right shape.

41. How did you make LLM calls reliable and cheap?

There are three layers in `llm_client.py`.

Caching: the key is a SHA-256 of the model name and the prompt, stored in `artifacts/llm_cache.json`. The same question to the same model never costs a second call, and runs are reproducible. That matters for tests and for a free-tier key with tight quotas.

```
def cache_key(model_name, prompt):
    joined = json.dumps([str(model_name), str(prompt)], ensure_ascii=False)
    return hashlib.sha256(joined.encode("utf-8")).hexdigest()
```

Retry with exponential backoff: three attempts, waiting 2s, then 4s. But I never retry `ModelNotConfigured`, because a missing API key won't fix itself and retrying would only waste time.

```
for attempt in range(attempts):
    try:
        return ask_model(prompt, cache_path)
    except ModelNotConfigured:
        raise
    except LanguageModelError:
```

```
        if attempt + 1 >= attempts:
            raise
        time.sleep(pause_seconds * (2 ** attempt))
```

Strict parsing: every prompt asks for JSON only. `read_answer` finds the outermost `{...}` (models like to wrap JSON in prose or code fences), parses it, and checks the required keys are present. Then the caller checks values against the published sets. For example, a tone outside calm, frustrated or angry is an error, not something I accept.

Every failure is a typed `LanguageModelError`. In batch mode it becomes a row in `batch_failures.csv`. In the API it becomes a 503.

Escalation then combines three independent signals (complaint older than 21 days, urgency is emergency, tone is angry) and escalates when at least two are present. Two of three is deliberate, so neither the model nor the LLM can escalate a complaint on its own.

Part 8: Retrieval (Weeks 6–7)

42. How did you chunk and index the tariff orders?

I load only the documents named in `tariff_orders_manifest.json`. A stray PDF in the folder is ignored. Text is extracted with `pypdf`, and an unreadable or empty PDF raises `UnreadableOrder` instead of silently indexing nothing.

Chunking uses fixed windows of 900 characters with 150 characters of overlap, so a sentence that straddles a boundary appears whole in at least one chunk. I guard against overlap ≥ size, because then the window would never move forward.

```
def split_into_chunks(text, size=900, overlap=150):
    if overlap >= size:
        raise ValueError("... so the cut would never advance")
    step, pieces, start = size - overlap, [], 0
    while start < len(text):
        piece = text[start:start + size]
        if piece.strip():
            pieces.append(piece)
        if start + size >= len(text):
            break
        start += step
    return pieces
```

Each chunk gets a stable ID, `<order_id>-000`, and metadata: `order_id, title, consumer_category, effective_from, effective_to, section`. The section is the first known heading found in the chunk. If a chunk has no heading, it inherits the previous chunk's section, because it's a continuation of it. Duplicate IDs raise an error.

Embeddings use `all-MiniLM-L6-v2` from sentence-transformers. It's small (about 90 MB), fast, and good enough for paragraph-level matching. They're stored in a persistent Chroma collection with cosine distance. Rebuilding deletes and recreates the collection, so it's idempotent.

43. How did you handle superseded orders?

The design decision is that **the date logic is deterministic metadata filtering, not semantic search**. Pure similarity search would happily return the 2024 order's fixed charge for a 2026 question, because the wording is almost identical. So I first resolve, from the manifest, the one order in force for that category on the first day of the billing period, and then restrict the vector search to that order with a `where` clause. I showed that code in answer 13.

Three tests prove the behaviour: the order returned is the one whose window contains the period, the same question in two periods reaches two different orders, and no chunk comes back from another category. A fourth test shows that

without the filter an out-of-force extract does come back, which justifies the whole mechanism. A period with no order in force is refused with `NoOrderInForce`. Answering from an order that had stopped having effect would be worse than not answering.

44. How do you stop the grounded answer from hallucinating?

There are three layers.

The prompt says: use only the extracts, cite every extract by its ID in square brackets, never cite an ID that wasn't handed over, and if the extracts don't answer, reply exactly `NOT IN THE ORDER` with no citations. Each extract is labelled with its ID, order, section and validity window.

Then I verify instead of trusting. I pull every `[ID]` out of the answer text and the citations list, and if any wasn't among the retrieved chunks, I reject the whole answer:

```
def check_citations(answer_text, chunks):
    handed_over = {chunk["chunk_id"] for chunk in chunks}
    cited = set(CITATION_PATTERN.findall(str(answer_text)))
    invented = sorted(cited - handed_over)
    if invented:
        raise UngroundedAnswer(f"the answer cites {', '.join(invented)}, which was never handed
over")
    return sorted(cited)
```

And if nothing was retrieved, I never call the model at all: `build_answer_prompt` raises `UngroundedAnswer` first.

Every question is written to `tariff_answers.csv` or `filtered_answers.csv` with the answer, the citations, the chunks handed over and any reason for refusal, so retrieval quality can be audited against the 148-question evaluation set.

Part 9: The agentic assistant (Weeks 7–8)

45. Walk me through the LangGraph assistant.

State. It's a `TypedDict` with the question, the consumer's category and period, the triage outputs, the plan, draft, answer, refusal, veto, held, a finished flag, and two accumulating channels, `steps` and `tool_calls`, which use `Annotated[List, operator.add]`. Each node returns only what it changed, and LangGraph appends to those two lists, so `steps` ends up being an exact record of the path the run took.

Nodes. There are seven, registered from a single tuple so nothing can be declared without being wired, or wired without being declared:

```
ASSISTANT_NODES = (("triage", triage_the_question), ("plan", plan_tools), ("tools", run_tools),
                   ("draft", draft_answer), ("critique", critique_draft),
                   ("gate", approval_gate), ("respond", write_response))
```

Triage, tools and gate are deterministic Python. Plan, draft, critique and respond are LLM agents, each with a narrow prompt and a strict JSON reply.

Edges. There are four conditional exits: after triage, tools, critique and gate. The other three edges are fixed:

```
graph.add_edge(START, "triage")
graph.add_conditional_edges("triage",   worth_answering, {CARRY_ON: "plan",    END: END})
graph.add_edge("plan", "tools")
graph.add_conditional_edges("tools",    still_going,     {CARRY_ON: "draft",   END: END})
```

```
graph.add_edge("draft", "critique")
graph.add_conditional_edges("critique", still_going,     {CARRY_ON: "gate",    END: END})
graph.add_conditional_edges("gate",     still_going,     {CARRY_ON: "respond", END: END})
graph.add_edge("respond", END)
```

The routers only read the state. They never call a model or re-read the question, because a router that did work would do it twice. `worth_answering` and `still_going` have identical logic today, but they're separate names because they sit at different points in the flow, and changing one shouldn't silently change the other.

Tools. There are four, all LangChain `@tool`s with Pydantic argument schemas: `look_up_reading` (SQLite), `calculate_bill` (the Week 1 function), `predict_reading_exception` (the champion plus operating rule, returning the class and its measured precision) and `search_tariff_order` (date-filtered retrieval). Expensive resources like the model, embedder and Chroma store are opened once and cached.

```
class BillRequest(BaseModel):
    consumer_category: str
    units_consumed: float = Field(ge=0)
    sanctioned_load_kw: float = Field(gt=0)          # an invented zero load is refused here

@tool(args_schema=BillRequest)
def calculate_bill(consumer_category, units_consumed, sanctioned_load_kw):
    """Price one billing cycle under the published schedule for its consumer category..."""
    amount = calculate_bill_amount(consumer_category, units_consumed, sanctioned_load_kw)
    return {..., "current_demand_inr": round(float(amount), 2)}
```

Safety. `run_tools` refuses a plan that names any unknown tool before running anything. The gate holds the run if a tool read the reading as `tamper_suspected` or `tariff_mismatch`, if the critic vetoed, or if nothing was drafted:

```
def approval_gate(state):
    reasons = [f"read as {named}" for named in _held_classes(state)]
    if str(state.get("veto") or "").strip():
        reasons.append(state["veto"])
    if not str(state.get("draft") or "").strip():
        reasons.append("nothing was drafted")
    if not reasons:
        return {"held": "", "steps": ["gate"]}
    return {"held": "; ".join(reasons), "answer": constants.HELD_FOR_A_PERSON_REPLY, "steps":
["gate"]}
```

Persistence and human-in-the-loop. The graph is compiled with a `SqliteSaver` checkpointer and `interrupt_before=["respond"]`. Every run pauses before responding. If it isn't held, the workflow resumes it immediately. If it is held, it stays paused under its thread ID until an inspector releases it, as in answer 12.

Exposure. All of this sits behind `POST /api/v1/assistant/questions` for any signed-in user, and `POST /api/v1/assistant/threads/{id}/release` for inspectors only. LLM failures become 503 and unknown threads become 404.

The phrasing of the replies varies from run to run, but the figures, the tools chosen, the class the champion read and the steps recorded don't. Those are the parts that must be reliable, and they're all deterministic.

Part 10: Follow-ups

46. How did you test it?

The repository has one grader file of 178 tests, three to five per day, and I made each day's tests pass before moving on. The style is behavioural:

- Week 1 scripts run as subprocesses with input piped in, and the printed figures are checked to the paisa.
- Boundary cases are tested deliberately: exactly 100 units vs 101 for the rebate cliff, one unit past a commercial block, rollover vs backwards.
- API routes are tested with FastAPI's `TestClient` against the in-memory repository, including every failure code.
- The assistant is tested with the graph's structure (nodes, edges, router outputs) and with stubbed agents, so the tests don't depend on a live LLM.
- Retrieval is tested for the four filtering properties above.

On top of that I wrote my own test files per week (`test_week*.py`).

47. How would you take it to production?

- **Storage:** move from SQLite to Postgres with Alembic migrations. Accounts are currently an in-memory dictionary, which is the first thing I'd move into the database.
- **Auth:** refresh tokens, key rotation, and the secret in a vault.
- **Rate limiting:** move it to Redis or the API gateway so it works across instances, since my fixed window lives in process memory.
- **Models:** an MLflow registry with versioned champions and operating rules, drift monitoring on the input features, and scheduled retraining.
- **Retrieval:** a managed vector database, plus re-ranking and hybrid keyword + vector search for exact figures.
- **Assistant:** Postgres checkpointer, async execution, tracing (LangSmith or OpenTelemetry) on every node, and a review queue UI for inspectors instead of a raw endpoint.
- **Batch jobs:** move them to a scheduler like Airflow, with the failure CSVs feeding alerts.

48. What would you do differently?

I'd move the tariff schedule out of `constants.py` and into versioned data keyed by effective date, the same way the tariff orders already are. Right now the pricing uses one schedule while retrieval knows about twelve periods. A bill for a cycle under an older order should be priced with that order's slabs, and proration already gives me the mechanism for a mid-cycle change.

I'd also consolidate the domain logic. The connection classes and the tariff functions both contain slab arithmetic. I'd have the classes delegate entirely to `tariff_calculations`, so there's only one implementation to keep correct.